

Laboratory Work No. 2

Constructing Models Based on Pre-trained Neural Networks

Student Name: [Your Name]

Group/ID: [Your Group]

Variant No.: [e.g., 2]

Dataset: [e.g., Fashion-MNIST]

Architecture: [e.g., ResNet18]

Instructor: Assoc. Prof. A. Kroshchanka

Date: June 24, 2026

Brest State Technical University
Department of Intellectual Information Technologies

Abstract

This report details the process of adapting and fine-tuning a pre-trained neural network using the PyTorch framework. The primary objective is to modify an existing architecture (e.g., ResNet, MobileNet) to fit a specific classification task and evaluate its performance. The report includes data preprocessing (such as handling single-channel images), architecture modification, performance comparison against custom models from Laboratory Work 1, state-of-the-art benchmarking, and model inference visualization on arbitrary images.

Contents

1	Introduction and Objectives	2
2	Variant Details	2
3	Data Preprocessing	2
4	Model Adaptation	3
5	Training and Results	3
5.1	Loss Curve	3
5.2	Evaluation and Comparisons	3
5.2.1	Pre-trained vs. Custom Architecture (Lab 1)	3
5.2.2	Comparison with SOTA	4
6	Inference Visualization	4
7	Conclusion	4
A	Assignment Variants	5

1 Introduction and Objectives

Laboratory Goal

To train neural networks constructed on the basis of pre-trained architectures using transfer learning techniques.

The specific tasks to be completed in this laboratory work are:

1. Organize the training process of a pre-trained neural network by adapting its layer structure to the assigned dataset. Use the same optimizer from Laboratory Work 1.
2. Plot the training/validation loss curves and evaluate the test set efficiency.
3. Compare the obtained results with the custom CNN architecture developed in Laboratory Work 1.
4. Review state-of-the-art (SOTA) results for the dataset and draw conclusions.
5. Visualize the inference of both the pre-trained and the custom network by feeding them an arbitrary image (e.g., from the Internet) and displaying the classification output.

2 Variant Details

According to the assignment, the experiment is conducted using the configuration in Table 1.

Table 1: Assigned Configuration

Parameter	Value
Variant Number	[Insert Variant, e.g., 2]
Dataset	[e.g., Fashion-MNIST]
Optimizer	[e.g., SGD]
Pre-trained Architecture	[e.g., ResNet18]
Loss Function	CrossEntropyLoss

3 Data Preprocessing

Pre-trained models in PyTorch typically expect input images to be 3-channel RGB tensors of a certain minimum size (often 224×224). For datasets containing small, single-channel images (like MNIST or Fashion-MNIST), data transformation is required.

The `torchvision.transforms` module was utilized to resize the images and convert grayscale inputs to 3-channel tensors by duplicating the channel:

```
1 import torchvision.transforms as transforms
2
3 # Transformation for 1-channel datasets (e.g., MNIST) to fit a 3-channel
  model
4 data_transform = transforms.Compose([
5     transforms.Resize(224), # Resize to match the pre-trained model
    input
6     transforms.Grayscale(num_output_channels=3), # Convert 1-channel to
    3-channels
7     transforms.ToTensor(),
8     transforms.Normalize(mean=[0.485, 0.456, 0.406], # ImageNet
    statistics
                                std=[0.229, 0.224, 0.225])
9 ])
10 ])
```

Listing 1: Data Transformation Pipeline

4 Model Adaptation

The assigned pre-trained architecture was loaded using the `torchvision.models` module. To adapt the model for the new dataset, the final fully connected layer (classifier) was replaced to match the number of target classes (e.g., 10 classes).

```
1 import torchvision.models as models
2 import torch.nn as nn
3
4 # Load pre-trained ResNet18
5 model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
6
7 # Modify the final fully connected layer
8 num_ftrs = model.fc.in_features
9 # Replace with a new linear layer (e.g., 10 output classes)
10 model.fc = nn.Linear(num_ftrs, 10)
```

Listing 2: Adapting a Pre-trained ResNet18

5 Training and Results

The adapted network was trained using the `CrossEntropyLoss` criterion and the designated optimizer.

5.1 Loss Curve

The graph demonstrates the convergence of the pre-trained model. Transfer learning allowed the model to reach a lower loss significantly faster than training from scratch.

5.2 Evaluation and Comparisons

5.2.1 Pre-trained vs. Custom Architecture (Lab 1)

The modified pre-trained architecture achieved an accuracy of **XX.X%** on the test set, compared to **YY.Y%** achieved by the custom CNN from Laboratory Work 1. The pre-

trained model demonstrated superior feature extraction capabilities due to weights initialized from a large-scale dataset (ImageNet).

5.2.2 Comparison with SOTA

While the pre-trained model achieves highly competitive results, absolute SOTA models on this dataset utilize complex training regimes, extensive data augmentation, and modern architectures (such as Vision Transformers). However, the transfer learning approach provided near-SOTA performance with a fraction of the computational cost.

6 Inference Visualization

An arbitrary image was sourced from the Internet to test the robustness of both models. The image was preprocessed, transformed into a tensor, and fed into both the Lab 1 custom CNN and the Lab 2 pre-trained model.

The pre-trained model outputted the correct classification with a high confidence score, demonstrating excellent generalization capabilities on unseen, real-world data.

7 Conclusion

This laboratory work successfully demonstrated the process of utilizing transfer learning by adapting a pre-trained neural network. Modifying the layer structure and applying proper data transformations allowed the model to effectively classify the target dataset. The comparison highlighted the significant advantages of pre-trained models over simple custom architectures in terms of accuracy and convergence speed.

References

- [1] PyTorch Documentation. *Models and pre-trained weights*. Retrieved from <https://pytorch.org/vision/stable/models.html>

A Assignment Variants

Each student is assigned a specific dataset, optimizer, and pre-trained architecture based on their variant number. The configuration for all 20 variants is presented in Table 2.

Table 2: Complete List of Assignment Variants for Laboratory Work 2

Variant No.	Dataset	Optimizer	Pre-trained Architecture
1	MNIST	SGD	AlexNet
2	Fashion-MNIST	SGD	ResNet18
3	CIFAR-10	SGD	ResNet34
4	CIFAR-100	SGD	MobileNet v3
5	STL-10 (labeled)	SGD	DenseNet121
6	MNIST	Adam	ResNet18
7	Fashion-MNIST	Adam	ResNet34
8	CIFAR-10	Adam	MobileNet v3
9	CIFAR-100	Adam	DenseNet121
10	STL-10 (labeled)	Adam	SqueezeNet 1.1
11	MNIST	Adadelata	ResNet34
12	Fashion-MNIST	Adadelata	MobileNet v3
13	CIFAR-10	Adadelata	DenseNet121
14	CIFAR-100	Adadelata	SqueezeNet 1.1
15	STL-10 (labeled)	Adadelata	MobileNet v2
16	MNIST	RMSprop	MobileNet v3
17	Fashion-MNIST	RMSprop	DenseNet121
18	CIFAR-10	RMSprop	SqueezeNet 1.1
19	CIFAR-100	RMSprop	MobileNet v2
20	STL-10 (labeled)	RMSprop	ShuffleNet v2